

# AI+金融财报分析系统完整全栈框架

## 一、核心思路与原则

利用大模型的能力对上市公司年报进行解读，从财务报表，行业性质等角度分析该企业年度经营情况，发展前景，投资价值。

整个系统涉及大模型、代码、数学计算，整体原则如下：

- 大模型只负责理解文字、标签分类、逻辑推演（商业常识）、检索研报。**AI不参与任何数学计算。**
- 代码只负责提取数字、换算单位、做勾稽校验、算MAD偏差、执行概率分配评分。**代码不进行任何语义理解。**
- 整体实现流程：**先看行业定性 -> 再看财务数据是否符合基准（横/纵） -> 发现异常偏差 -> 结合财报原文和外部研报找原因 -> 给出概率化的综合判断。

## 二、系统六层架构详解

### ◆ 第0层：PDF数据提取

- 任务：**把一份几百页的上市公司年度报告PDF，拆解成电脑能直接读懂的 `JSON` 格式文件。
- 实现方式：**使用开源工具（如 `MinerU`）自动解析和切分。
- 最终输出结果：**一份包含四大区块的 `JSON` 文件：
  - 财务数据部分（三大报表的表格原始数据，通常是OCR识别后的文本格式）；
  - 管理层讨论与分析部分（全是文字，描述公司去年的经营情况、风险等）；
  - 附注明细部分（表格辅助信息，比如存货的具体构成）；
  - 公司基本情况部分（公司的主营业务介绍）。

### ◆ A层：找对比基准（行业定性）

- A0层：外部宏观研报提取**
  - 任务：**大模型利用联网搜索，搜索该公司的行业宏观情况，上下游供应商等（比如：“光伏行业 2025年上半年 硅料价格走势”，“房地产下游 销售景气度”）。
  - 实现方式：**输入设定好的搜索词，强制大模型只提取“客观事实”（价格涨跌幅、供需情况），**屏蔽**掉研报里的“强烈看好/强烈卖出”等主观推荐语。
  - 输出结果：**一份简短的外部宏观背景事实。
- A1层：语义提取与打标签**
  - 任务：**大模型只读 `JSON` 里的文字部分（公司基本情况和行业背景）。给这家公司打上关键的**定性标签**。
  - 输出结果：**
    - 硬标签**（比如：同花顺三级行业标签-“白酒”）。
    - 软标签**（比如：“重资产”、“高毛利”、“To-C端”、“依靠品牌护城河”）。
- A2层：匹配模式相似的企业**
  - 任务：**拿到A1的标签后去数据库里找相似企业
  - 第一步（强制约束）：**先按**硬标签**锁定行业范围（必须从“白酒”行业里找同行）。
  - 第二步（动态滤网）：**在“白酒”行业里，再用**软标签**去过滤，找到“商业模式最相似的前 3~5 家公司”。

- **第三步（提取基准）：**

- 算出这3~5家同行各项财务指标的**中位数**，作为**横向比较基准**。
- 提取**这家公司自己**过去5年（剔除异常暴雷年份）的数据均值，作为**纵向比较基准**。

## ◆ B层：提取财务数据

- **B0层：表头语义指引**

- **任务：**大模型只看财务报表的表头文字。分辨表格名称和单位。
- **输出结果：**
  - 统一字段名（比如把“营业收入”、“营业总收入”统一映射为系统标准字段 `Revenue_Total`）；
  - 提取单位（元/万元/千元）；
  - 提取报表类型（合并报表 / 母公司报表）。

- **B1层：代码准确提取表格数据与勾稽校验**

- **任务：**纯代码（Pandas 或表格解析库）根据B0指引，定位到表格的具体行和列，抓取数字。
- **单位换算：**把单位统一换算成“元”（比如原来单位是万元，抓出数字后代码自动乘10000）。
- **勾稽关系校验：**代码提取完毕后，校验最基础的会计恒等式：
  - `资产总计` 是否等于 `负债总计` + `所有者权益总计`
  - `期末未分配利润` 是否约等于 `期初未分配利润` + `本期净利润` - `本期分红` 等等
- **分支处理：**
  - **情况A（校验失败）：**直接跳过下面所有步骤，在E层输出  
【系统异常阻断：PDF数据源损坏或低劣造假，请人工核验原文件】。
  - **情况B（校验成功）：**数据传给下一层。同时把母公司报表的货币资金、负债数据单独提取备用。

## ◆ B+层：内部逻辑检查（造假初步判断）

- **任务：**用代码查财报数据自己有没有“逻辑硬伤”。
- **具体计算3项指标：**
  - 净现比/收现比：**算 `经营活动现金流净额 / 净利润` 和 `销售商品收到的现金 / 营业收入`。如果两个比值都显著低于 0.6。
  - 存贷双高：**算 `货币资金总额` 和 `有息负债总额`。两者如果同时处于高位。
  - 母子资金分离度：**算 `合并报表货币资金 - 母公司报表货币资金`。如果差值极大。
- **输出结果：**列出异常 { "逻辑异常清单": ["存贷双高可疑", "利润含金量极低"] }。但不下最终定论，最终判断交给D层。

## ◆ C层：算偏差（抓出反常指标）

- **任务：**不去判断好坏，只用代码算“这家公司的指标偏离平均水平多远”。
- **算法（采用MAD）：**因为财务数据常有极端暴雷值，所以采用统计学上更稳的 **MAD（中位数绝对偏差）** 算法。
- **输出结果：**一份异常波动清单，例如  
[指标：存货周转率，实际值：0.1，偏离度：低于同行中位数 3.2倍 MAD]。

## ◆ D层：原因查找与分配概率（利用大模型实现）

- **任务：**对于每一个异常，通过以下方式分析原因
- **D1层：双路径推理**（两条路互不越界）
  - **路1（查原文）：**大模型只去读年报的【附注】和【管理层讨论】原文，提取其中的客观解释（如果原文含糊其辞，标记为“无直接解释”）。

- **路2（结合常识推演）**：大模型不去看原文，而是拿 **A0层的宏观研报事实 + A1的行业标签 + C层（或B+层）的偏离度（或异常）**，发散出几种合理的假设（比如：假设A是宏观原材料跌价导致备货，假设B是产品滞销）。
- 利用上面的两路先分析B+层的异常，B+层基本上只存在两种情况：行业本身性质或造假（比如说存贷双高对于房地产以及多子公司来说是本身性质），再分析C层的偏差。
- **D2层：概率分配**
  - **任务**：把D1的两条路结合起来，给出具体的概率。
  - **举例**（对于某一个异常）：
    - 对于C，如果路1有明确战略解释 + 路2推演无冲突 => **70%策略调整，10%造假，20%其他**
    - 对于B+，如果路1无解释 + 路2指向行业本身性质 => **100%行业本身性质**
  - **最终输出**：如 [异常指标：存货周转率] => [归因：经营滞销(60%)，战略备货(30%)，其他(10%)]。

## ◆ E层：最终输出打分与报告

- **E1层：综合置信度打分**
  - **基础分**：100分。
  - **扣分机制（双层扣分）**：基于D的最终结果进行打分
    - C层扣分（偏离度扣分）**：首先，系统内建一个权重库，按照影响程度映射：极其严重（如财务造假）-3，有些严重（如经营恶化）-1，正向利好 +1。基于此进一步加入偏离动态变化：
      - **实际影响权重 = 基础影响权重 × (1 + MAD偏离度 / 3)**，理由：如果‘存货周转率’偏离度只是 1.5 倍 MAD，我们按照基础 -1 权重计算。但如果它偏离了 6 倍 MAD（极其离谱），说明该异常极其严重，如果我还给 -1 权重，就明显低估了潜在伤害。因此，我们引入偏离度放大系数，偏离越大，影响权重越高。
    - 然后对于D列出的每一条属于C的部分，算 **期望影响 = 概率 \* 实际影响权重**。最后 **C层扣分 = 期望影响 \* MAD偏离度**。
    - B+层扣分**：行业性质赋予权重0。造假赋予权重-3。其次，算**严重程度评分**进行修正。
    - 修正系数（严重程度）的考量如下：
      - 存贷双高严重程度 = (货币资金/总资产 \* 有息负债/总资产) 的乘积。如果数值高于行业通常水平（比如 > 0.3），严重程度 = 5。如果是 0.1~0.3，严重程度 = 2~3。
      - 净现比/收现比严重程度 = 正常阈值0.6。如果 实际净现比 = 0.4，严重程度 = (0.6 - 0.4) / 0.2 \* 2 = 2分。如果 实际净现比 = -0.5（经营现金流为负数），严重程度直接锁死为最高分 5分。
      - 母子资金分离度严重程度 = (合并报表货币资金 - 母公司货币资金) / 总资产。如果这个比例 > 30%，严重程度 = 5；> 15%，严重程度 = 3。
      - 最后 **B+层扣分 = 权重 \* 修正系数**。
  - **最终得分 = 100 - (C层扣分 + B+层扣分)**。
- **E2层：生成结构化报告**
  - 输出四个模块：
    - 【整体研判】**：E1给出的综合得分，并列相似企业横向对比。
    - 【核心异常与概率归因】**：列出B+和C层提取的异常指标、D层算出的概率分布。最后再展示判断依据（标出该推理依据的年报页码或附注名称）。
    - 【多空逻辑栈】**：最后展示“看多支持点”和“看空风险点”。

```
ai-finance-analyzer/
├── README.md
├── pyproject.toml          # 项目元信息与依赖管理
├── .env.example           # API Key 等敏感配置模板
├── .gitignore
├── config/                # ⚙️ 全局配置（不变的数据）
│   ├── weights.yaml      # E层权重库：各指标的基础影响权重
│   ├── industry_tags.yaml # A层行业标签体系
│   ├── thresholds.yaml   # B+/C层 异常判定阈值
│   └── prompts/          # 所有 LLM prompt 模板
│       ├── a0_macro_search.yaml
│       ├── a1_tagging.yaml
│       ├── d1_explain.yaml
│       └── ...
├── schemas/               # 📄 跨层共享的数据契约（Pydantic）
│   ├── __init__.py
│   ├── raw_doc.py        # 第0层输出：JSON 四大区块的定义
│   ├── financial.py      # 统一财务字段体系（Revenue_Total 等）
│   ├── tags.py           # 硬标签 + 软标签
│   ├── anomaly.py        # B+异常清单 / C层MAD偏离度
│   └── report.py         # E层最终报告结构
├── pipeline/              # 🔗 全流程编排
│   ├── __init__.py
│   ├── orchestrator.py   # 按顺序调度 0→A→B→B+→C→D→E
│   ├── parallel_runner.py # 可并行的步骤（如A0+A1）
│   └── state.py           # 各层之间的数据传递上下文
├── layers/                # 🌱 六层实现
│   ├── __init__.py
│   ├── layer_0_extract/   # PDF → JSON
│   │   ├── pdf_parser.py
│   │   └── chunker.py
│   ├── layer_a_benchmark/ # 行业定性 & 找基准
│   │   ├── a0_macro_search.py # LLM 联网搜研报
│   │   ├── a1_tagging.py      # LLM 打标签
│   │   └── a2_matcher.py      # 纯代码：匹配相似企业 + 算基准
│   ├── layer_b_extract/   # 财务数据提取（分割！）
│   │   ├── b0_semantic_guide.py # LLM：读表头，做字段映射
│   │   ├── b1_extractors.py    # 纯代码：定位表格、取数、换算
│   │   └── b1_validators.py    # 纯代码：勾稽关系校验
│   ├── layer_bplus_internal/ # 造假初筛（纯代码）
│   │   └── logic_checks.py     # 净现比 / 存贷双高 / 母子分离
│   ├── layer_c_deviation/ # MAD 偏差计算（纯代码/数学）
│   │   └── mad_calculator.py
│   └── layer_d_explain/   # 异常判定（纯代码/规则引擎）
│       └── d1_explain.py
```

```

└─ deviation_scorer.py

└─ layer_d_reasoning/      # 原因查找 + 概率分配 (LLM主力)
  └─ d1_dual_path.py      # 双路径推理编排
  └─ d1_lookup_notes.py   # 路1: 查年报附注/管理层讨论
  └─ d2_probability.py    # 概率分配

└─ layer_e_output/        # 最终打分 & 报告
  └─ e1_scoring.py        # 纯代码: 置信度打分
  └─ e2_report_gen.py     # 结构化报告生成

llm/                      # 🤖 LLM 调用封装 (统一网关)
└─ __init__.py
└─ client.py              # 统一的 LLM 调用接口 (支持多厂商)
└─ prompt_loader.py       # 从 config/prompts/ 加载模板
└─ response_parser.py     # 解析 LLM 输出为结构化数据

data/                     # 📦 数据 (git-ignored 或 DVC 管理)
└─ raw/                   # 原始 PDF
└─ processed/             # 第0层输出的 JSON
└─ benchmarks/            # 行业基准数据库 (同行指标中位数)
└─ outputs/               # 最终报告

tests/                    # 🧪 测试
└─ test_layers/
  └─ test_b1_validators.py
  └─ test_c_mad.py
  └─ test_e1_scoring.py
└─ test_llm/
  └─ test_prompt_loader.py
└─ fixtures/              # 测试用 mock 数据
  └─ sample_财报.json
  └─ mock_benchmarks.csv

notebooks/                # 📓 探索性分析
└─ prototype_deviation.ipynb

scripts/                  # 🛠️ 运维/工具脚本
└─ run_pipeline.py        # 命令行入口: python run_pipeline.py --pdf xxx.pdf
└─ build_benchmark_db.py  # 初始化行业基准数据库
└─ validate_pdf.py        # 快速验证 PDF 可解析性

```